

Final y abstractos en Java

Dos palabras que deciden qué se puede cambiar y qué está obligado a existir.

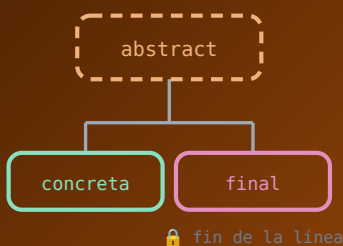
Aprende a **cerrar con llave** lo que no debe tocarse y a **dejar el molde abierto** para que otros lo completen.

Nivel: iniciación

8 sesiones

+18 ejemplos de código

3 laboratorios



SESIÓN 01 · CONCEPTO FUNDAMENTAL

Dos palabras, dos intenciones opuestas

Una cierra puertas; la otra deja huecos que alguien debe llenar

Cuando programas en Java no solo escribes *qué hace* tu código: también decides **quién puede cambiarlo después**. Esa decisión se expresa con dos palabras clave que aprenderás en esta guía.

final significa "esto queda así, no se toca". **abstract** significa lo contrario: "esto está incompleto a propósito, alguien tiene que terminarlo".

Entender cuándo usar cada una es lo que separa un código que otros pueden mantener de uno que se rompe cada vez que alguien lo modifica.

i Definición rápida

final → sella. Impide reasignar un valor, sobrescribir un método o heredar de una clase.

abstract → obliga. Declara algo sin implementarlo, forzando a las clases hijas a completarlo.

★ ¿Sabías qué?

Estas dos palabras son **mutuamente excluyentes**. Nunca verás `abstract final` junto: sería pedirle a alguien que complete algo que prohibiste modificar. El compilador lo rechaza de inmediato.

La metáfora que usaremos: la casa en construcción

Imagina un **plano arquitectónico**. Algunas cosas están decididas y firmadas: la ubicación del terreno, el número de la casa, los cimientos. Nadie puede cambiarlas. Eso es **final**.

Otras están dibujadas pero vacías: "aquí va una habitación", "aquí va un baño". El plano *exige* que existan, pero deja que cada dueño decida el color, los muebles y el acabado. Eso es **abstract**.

final = firmado y sellado

abstract = hueco obligatorio

clase hija = quien termina la obra

herencia = construir sobre el plano

Panorama visual de lo que viene

```
// Las tres puertas donde puedes poner "final"

final int edad = 18;           ← en una PROPIEDAD: el valor no cambia
final void saludar() {...}    ← en un MÉTODO: nadie lo sobrescribe
final class Config {...}      ← en una CLASE: nadie hereda de ella

// Y el contrato que "abstract" impone

abstract class Figura {
    abstract double area();    ← sin cuerpo: la hija DEBE escribirlo
}
```

Tu número de cédula

Te lo asignan una vez y ya nunca cambia. Es un dato **final**.

Un plano de vivienda

Exige que haya cocina, pero no dice de qué color. Es **abstract**.



SESIÓN 02 · EL NIVEL MÁS BÁSICO

final en una propiedad

Un valor que se asigna una vez y jamás cambia

Una **propiedad** (también llamada atributo o campo) es una variable que vive dentro de una clase. Cuando la marcas como **final**, le dices a Java: "este valor se asigna una sola vez; después, cualquier intento de cambiarlo es un error".

Primer ejemplo, anotado línea por línea

```
Estudiante.java  JAVA

1 public class Estudiante {
2     ❶ private final String codigo;
3     private String nombre;
4     ❷ public Estudiante(String codigo) {
5         this.codigo = codigo;
6     }
7     ❸ public void cambiarNombre(String n) {
8         this.nombre = n; // permitido
9     }
10 }
11
12 }
```

- 1 La propiedad código es **final**: se le puede dar valor una vez y nunca más.
- 2 El **constructor** es el lugar perfecto para asignarla. Aquí todavía es legal.
- 3 `nombre` no es final, así que este método puede cambiarlo sin problema.

¿Qué pasa si intentas cambiarlo?

```
Prohibido.java  ERROR

1 public void hackear() {
2     this.codigo = "OTRO";
3 }
```

```
compilador → cannot assign a value to final
variable codigo
```

Lo importante

El error aparece **al compilar**, no cuando el programa ya está corriendo. Java te protege antes de que el problema llegue al usuario final.

Las constantes: static final

Cuando un valor es final y además es el mismo para todos los objetos de la clase, se combina con static. Así nacen las **constantes**.

```
Circulo.java  JAVA
1 public class Circulo {
2     ❶ public static final double PI = 3.14159;
3     ❷ private final double radio;
4     public Circulo(double radio) {
5         this.radio = radio;
6     }
7
8     ❸ public double area() {
9         return PI * radio * radio;
10    }
11 }
```

- ❶ static final = una constante compartida por toda la clase. Se escribe en **MAYÚSCULAS** por convención.
- ❷ radio es final pero **no** static: cada círculo tiene el suyo, y ninguno cambia después de creado.
- ❸ El método solo lee los valores; no intenta modificarlos.

! Antipatrón frecuente

Declarar final una propiedad y **olvidar asignarla** en el constructor. Java exige que toda propiedad final tenga valor antes de que el constructor termine.

✓ Buena práctica

Marca como final toda propiedad que **no deba cambiar** después de crear el objeto (identificadores, fechas de creación, configuraciones).

! La trampa más común: final no significa "immutable"

Si la propiedad final apunta a un **objeto**, lo que queda sellado es *la referencia*, no el contenido. Puedes seguir modificando lo que hay adentro.

```
Trampa.java  JAVA
1 private final List<String> notas = new ArrayList<>();
2 notas.add("Excelente"); // ✓ PERMITIDO: cambio el contenido
3 notas = new ArrayList<>(); // ✗ ERROR: cambio la referencia
```

? Punto de control

Si escribes final int[] numeros = {1, 2, 3}; — ¿puedes hacer numeros[0] = 99;?

► [Ver respuesta](#)



Laboratorio 1 — La tarjeta de identificación

Objetivo: distinguir qué datos deben sellarse y cuáles no.

1. Crea la clase Tarjeta con tres propiedades: numero, titular y saldo.
2. Marca como final **solo** las que no deberían cambiar nunca. Piensa: ¿cuál de las tres varía con el tiempo?
3. Asigna las propiedades final dentro del constructor.
4. Agrega una constante public static final String BANCO = "Banco Central";
5. Crea un método depositar(double monto) que modifique el saldo.

Reto extra: intenta escribir un método que cambie el numero. Compila y lee con calma el mensaje de error: es tu mejor profesor.



final en un método

Nadie puede cambiar cómo funciona esto

Normalmente, una **clase hija** puede reescribir los métodos que hereda. A eso se le llama **sobrescribir** (en inglés, *override*). Pero a veces hay comportamientos que *no deben* alterarse: una validación de seguridad, un cálculo legal, una regla del negocio.

Marcar un método como **final** bloquea esa posibilidad.

```
CuentaBancaria.java  JAVA
1 public class CuentaBancaria {
2     ❶ public final boolean validarClave(String c) {
3         return c != null && c.length() >= 8;
4     }
5     ❷ public void mostrarSaldo() {
6         System.out.println("Saldo disponible");
7     }
8 }
```

- 1 La validación de clave es **final**: ninguna subclase puede debilitarla ni saltársela.
- 2 Este método no es final, así que una subclase sí puede personalizarlo.

```
CuentaHackeada.java  ERROR
1 class CuentaHackeada extends CuentaBancaria {
2     @Override
3     public boolean validarClave(String c) {
4         return true; // ¡acepta cualquier clave!
5     }
6 }
```

compilador → validarClave() in CuentaHackeada
cannot override final method

```
CuentaAhorro.java  JAVA
1 class CuentaAhorro extends CuentaBancaria {
2     @Override
3     public void mostrarSaldo() {
4         System.out.println("Ahorro + interés");
5     }
6 }
```

Este sí compila: mostrarSaldo() nunca fue sellado.

Diagrama: qué se hereda y qué se bloquea

```
CuentaBancaria
final validarClave()  🔒 sellado
mostrarSaldo()      .. abierto
|
▼ extends
CuentaAhorro
validarClave()      ❌ NO puede reescribirlo
mostrarSaldo()      ✅ SÍ puede reescribirlo
```

También existe final en parámetros

Puedes sellar un parámetro para garantizar que no se reasigne dentro del método. Es una forma de proteger la claridad del código.

```
Calculadora.java  JAVA
1 public int duplicar(final int valor) {
2     // valor = 10; ❌ error: no se puede reasignar
3     return valor * 2; // ✅ leerlo siempre es válido
4 }
```

🚒 Salida de emergencia

El protocolo de evacuación no lo puede cambiar cada empleado: es **final**.

📄 Cálculo de impuestos

La fórmula está fijada por ley; ninguna sucursal puede reescribirla.

🎨 Color de la fachada

Sí se puede cambiar por sede: método normal, sobrescribible.

🔒 Verificación de identidad

Sellada para que ninguna subclase la debilite.

? Punto de control

¿Un método `private` necesita marcarse como `final` para que no lo sobrescriban?

► [Ver respuesta](#)



SESIÓN 04 · EL SELLO MÁS FUERTE

final en una clase

Fin de la línea: de aquí nadie hereda

Marcar una clase como `final` significa que **ninguna otra clase puede extenderla**. Es el nivel más restrictivo: sellas la clase completa, con todos sus métodos de una sola vez.

```
ConfiguracionSistema.java  JAVA
1  ① public final class ConfiguracionSistema {
2    ② public static final String VERSION = "2026.1";
3
4    public String obtenerVersion() {
5        return VERSION;
6    }
7 }
8
9  ③ // class Otra extends ConfiguracionSistema { }
10 // ✗ cannot inherit from final class
```

- 1 `final class` cierra la puerta a la herencia por completo.
- 2 Los métodos de adentro **no necesitan** marcarse `final`: ya nadie puede heredarlos para sobrescribirlos.
- 3 Cualquier intento de `extends` se detiene en compilación.

★ ¿Sabías qué?

La clase `String` que usas todos los días es **final**. Si cualquiera pudiera extenderla y alterar su comportamiento, se rompería la seguridad de millones de programas. Lo mismo ocurre con `Integer`, `Double` y todas las clases envoltorio.

Un camino intermedio en Java moderno: `sealed`

A veces `final` es demasiado estricto y la herencia abierta es demasiado floja. Java moderno ofrece una opción intermedia: **clases selladas**, que permiten herencia pero *solo a una lista autorizada*.

```
Vehiculo.java  JAVA
1  ① public sealed class Vehiculo
2      permits Carro, Moto {
3      protected String placa;
4  }
5
6  ② final class Carro extends Vehiculo { }
7  ③ non-sealed class Moto extends Vehiculo { }
```

- 1 `sealed ... permits` dice exactamente **quiénes** pueden heredar. Nadie más.
- 2 `Carro` es `final`: la cadena termina aquí.
- 3 `non-sealed` reabre la puerta: de `Moto` sí puede heredar cualquiera.

Modificador	¿Se puede heredar?	¿Quién puede?	Cuándo usarlo
<code>class</code>	Sí	Cualquiera	Caso general, jerarquías abiertas
<code>sealed</code>	Sí, restringido	Solo las listadas en <code>permits</code>	Grupos cerrados y conocidos de subtipos
<code>final</code>	No	Nadie	Clases de utilidad, valores, seguridad

! Antipatrón

Marcar **todas** tus clases como `final` "por si acaso". Bloqueas la reutilización y haces imposible extender tu código cuando el proyecto crezca.

✓ Buena práctica

Usa `final` cuando la clase representa un **valor cerrado** o contiene lógica sensible. En los demás casos, deja la puerta abierta.

? Punto de control

¿Puede una clase `final` heredar de otra clase?

► [Ver respuesta](#)



SESIÓN 05 · EL MOLDE INCOMPLETO

Clases abstractas

Un plano que no se puede habitar, pero que guía toda la construcción

Una **clase abstracta** es una clase que **no se puede instanciar**. No puedes crear objetos directamente de ella. Existe únicamente para que otras clases hereden de ella y la completen.

i La idea clave

"Animal" es un concepto real, pero **no existe un animal genérico** caminando por ahí. Existen perros, gatos y aves. `Animal` es la clase abstracta; las otras son las concretas.

★ ¿Sabías qué?

Una clase abstracta **sí puede tener constructor**, aunque no puedas instanciarla. Ese constructor lo llaman las clases hijas mediante `super()` para inicializar lo que heredan.

```

Animal.java
JAVA
1  ① public abstract class Animal {
2    ② protected final String nombre;
3    ③ public Animal(String nombre) {
4      this.nombre = nombre;
5    }
6    ④ public void dormir() {
7      System.out.println(nombre + " duerme.");
8    }
9  }
10
11 }
```

- 1 `abstract class`: no se puede hacer `new Animal(...)`.
- 2 Puede tener propiedades normales, incluso `final`.
- 3 Tiene constructor, usado por las hijas con `super(nombre)`.
- 4 Puede tener métodos **completos**, no solo vacíos. Todas las hijas los heredan listos.

```

✗ Imposible.java
ERROR
1  Animal a = new Animal("Genérico");

compilador → Animal is abstract; cannot be
instantiated
```

```

✓ Correcto.java
JAVA
1  Animal a = new Perro("Firulais");
2  a.dormir();

salida → Firulais duerme.
```

✓ Detalle poderoso

Fíjate en `Animal a = new Perro(...)`: la **variable** es del tipo abstracto, pero el **objeto** es concreto. Esto te permite tratar a todos los animales por igual sin importar cuál sea. Se llama **polimorfismo**.

Jerarquía visual



Solo las cajas verdes de abajo pueden convertirse en objetos reales.



SESIÓN 06 · EL CONTRATO OBLIGATORIO

Métodos abstractos

Declarados sin cuerpo: la hija está obligada a escribirlos

Un **método abstracto** se declara con su nombre y sus parámetros, pero **sin llaves y sin código adentro**. Termina en punto y coma. Es una promesa: "toda clase que herede de mí tendrá este método, aunque cada una lo haga a su manera".

abstract public double calcularArea();

① abstract

Marca que **no tiene cuerpo** y que las hijas deben implementarlo.

② tipo de retorno

Lo que el método devolverá cuando alguien lo implemente.

③ nombre y parámetros

La firma que **toda hija** debe respetar exactamente.

④ punto y coma

Sin `{ }`. Poner llaves aquí es un error de compilación.

Ejemplo completo, paso a paso

Figura.java

JAVA

```
1 public abstract class Figura {
2     ① public abstract double calcularArea();
3     ② public void describir() {
4         System.out.println("Área: " + calcularArea());
5     }
6 }
7 }
```

- 1 Método abstracto: sin cuerpo. Figura no sabe cómo calcular el área porque depende de la forma.
- 2 Método concreto que **llama** al abstracto. Funciona porque, en tiempo de ejecución, siempre habrá una implementación real.

Cuadrado.java

JAVA

```
1 class Cuadrado extends Figura {
2     private final double lado;
3     Cuadrado(double l) { lado = l; }
4     @Override
5     public double calcularArea() {
6         return lado * lado;
7     }
8 }
9 }
```

Circulo.java

JAVA

```
1 class Circulo extends Figura {
2     private final double radio;
3     Circulo(double r) { radio = r; }
4     @Override
5     public double calcularArea() {
6         return Math.PI * radio * radio;
7     }
8 }
9 }
```

Main.java

JAVA

```
1 Figura[] figuras = { new Cuadrado(4), new Circulo(2) };
2 for (Figura f : figuras) {
3     f.describir();    // cada una responde a su manera
4 }
```

salida → Área: 16.0 · Área: 12.566370614359172

! Error clásico del principiante

Si una clase hija **no implementa** todos los métodos abstractos que hereda, el compilador la rechaza. Solo hay dos salidas: implementarlos todos, o declarar también la hija como `abstract`.

Reglas que debes memorizar

Combinación	¿Válida?	Por qué
<code>abstract</code> + método sin cuerpo	✓ Sí	Es exactamente su propósito
<code>abstract</code> + <code>final</code>	✗ No	Uno exige sobrescribir, el otro lo prohíbe
<code>abstract</code> + <code>private</code>	✗ No	La hija no lo vería, no podría implementarlo
<code>abstract</code> + <code>static</code>	✗ No	Los métodos static no se sobrescriben
Clase abstracta sin métodos abstractos	✓ Sí	Raro, pero legal: impide instanciar

? Punto de control

¿Por qué `abstract final void metodo();` nunca compila?

► [Ver respuesta](#)

¿Clase abstracta o interfaz?

Ambas declaran métodos sin cuerpo, así que es normal confundirlas. Esta tabla te ayuda a decidir cuál usar.

Criterio	Clase abstracta	Interfaz
Se usa con	<code>extends</code>	<code>implements</code>
¿Cuántas se pueden heredar?	Solo una	Varias a la vez
Propiedades con estado	Sí, normales y <code>final</code>	Solo constantes
Constructor	Sí tiene	No tiene
Cuándo elegirla	Hay comportamiento y datos compartidos	Solo se define un contrato de capacidades

✓ Regla práctica para decidir

Pregúntate: ¿mis clases **"son un"** tipo de algo que comparte datos y código? → clase abstracta. ¿Solo **"pueden hacer"** algo, sin parecerse entre sí? → interfaz. Un Perro *es un* Animal, pero *puede* ser Adiestrable.



Laboratorio 2 — El zoológico

Objetivo: crear una jerarquía abstracta funcional desde cero.

1. Crea la clase abstracta `Animal` con la propiedad final `String nombre` y su constructor.
2. Agrega el método abstracto `public abstract String emitirSonido();`
3. Agrega un método concreto `presentarse()` que imprima el nombre y llame a `emitirSonido()`.
4. Crea `Perro`, `Gato` y `Vaca`, cada una implementando su propio sonido.
5. En el `main`, crea un arreglo `Animal[]` con los tres y recórrelo llamando a `presentarse()`.

Reto extra: intenta crear una clase `Pez` que *no* implemente `emitirSonido()`. Lee el error y luego arréglalo de las dos formas posibles.



SESIÓN 07 · VARIAS VERSIONES DEL MISMO MÉTODO

Sobrecarga de métodos abstractos

Mismo nombre, distintos parámetros, distintas obligaciones



Primero: no confundas estos dos conceptos

Sobrecarga (*overload*) = varios métodos con el **mismo nombre** pero **parámetros diferentes**, en la misma clase.

Sobrescritura (*override*) = una clase hija reescribe un método heredado con la **misma firma exacta**.

Criterio	Sobrecarga (overload)	Sobrescritura (override)
Dónde ocurre	En la misma clase	Entre clase padre e hija
Parámetros	Deben ser distintos	Deben ser idénticos
Anotación	No usa <code>@Override</code>	Usa <code>@Override</code>
Se decide	Al compilar	Al ejecutar

Sobrecargando métodos abstractos

Una clase abstracta puede declarar **varias versiones abstractas** del mismo método. Cada versión es un contrato independiente, y la clase hija está obligada a implementar **todas**.

```
Reporte.java  JAVA
1 public abstract class Reporte {
2     ❶ public abstract void generar();
3     ❷ public abstract void generar(String titulo);
4     ❸ public abstract void generar(String t, int p);
5 }
```

- 1 Versión sin parámetros: el reporte más simple.
- 2 Mismo nombre, **un** parámetro. Java las distingue por la lista de parámetros.
- 3 Tercera versión con dos parámetros. La hija deberá implementar **las tres**.

```
1 public class ReportePDF extends Reporte {
2     @Override public void generar() {
3         generar("Sin título", 1); // reutiliza la versión completa
4     }
5     @Override public void generar(String titulo) {
6         generar(titulo, 1);
7     }
8     @Override public void generar(String t, int pags) {
9         System.out.println("PDF: " + t + " (" + pags + " pág.)");
10    }
11 }
```

salida → PDF: Ventas 2026 (12 pág.)

✓ Patrón muy usado

Fíjate cómo las versiones cortas **delegan** en la más completa. Así escribes la lógica real una sola vez y evitas duplicar código.

! Esto NO es sobrecarga válida

Cambiar únicamente el **tipo de retorno** no distingue dos métodos. `void generar()` y `String generar()` chocan: Java no sabría cuál elegir.

☕ Pedir un café

"Un café", "un café grande", "un café grande con leche": mismo verbo, distinta información. Eso es sobrecarga.

🖨 Imprimir

Puedes imprimir un texto, una imagen o un documento completo: un solo nombre, varias firmas.

? Punto de control

Si la clase abstracta declara 3 versiones sobrecargadas y la hija implementa solo 2, ¿compila?

► [Ver respuesta](#)

⚡ Reto

Diseña una clase abstracta `Mensaje` con tres versiones sobrecargadas de `enviar()`: sin parámetros, con destinatario, y con destinatario más prioridad. Implementa `MensajeEmail` haciendo que las dos primeras deleguen en la tercera.



Buenas prácticas, repaso y autoevaluación

Lo que debes recordar cuando cierres este documento

✓ Buena práctica	✗ Antipatrón
Marcar <code>final</code> las propiedades que no deben cambiar tras crear el objeto	Dejar todo mutable "por si acaso" y perder el control del estado
Sellar con <code>final</code> métodos con lógica de seguridad o reglas de negocio	Marcar <code>final</code> absolutamente todo, bloqueando la reutilización
Usar clases abstractas cuando hay comportamiento compartido real	Crear jerarquías abstractas para conceptos que no comparten nada
Escribir siempre <code>@Override</code> al implementar un método abstracto	Omitirlo y no darte cuenta de que escribiste mal la firma
Nombrar las constantes <code>static final</code> en MAYÚSCULAS	Mezclar estilos de nombres y confundir a quien lea el código

Mapa mental

final en propiedad

Se asigna una vez. Protege la referencia, no el contenido del objeto.

final en método

Ninguna subclase puede sobrescribirlo. Sella comportamiento crítico.

final en clase

Nadie puede heredar. Fin de la jerarquía hacia abajo.

abstract

No se instancia. Obliga a las hijas a completar los huecos.

instanciar = crear un objeto con `new`

sobrescribir = reescribir método heredado

sobrecargar = mismo nombre, otros parámetros

firma = nombre + parámetros

polimorfismo = un tipo, muchas formas



Laboratorio 3 — Sistema de empleados

Objetivo: integrar `final`, clases abstractas, métodos abstractos y sobrecarga en un solo ejercicio.

1. Crea la clase abstracta `Empleado` con `final String cedula`, `String nombre` y la constante `static final double SALARIO_MINIMO`.
2. Declara el método abstracto `public abstract double calcularSalario();`
3. Sobrecárgalo con una segunda versión: `calcularSalario(int horasExtra)`.
4. Agrega un método `final void mostrarCedula()` que ninguna subclase pueda alterar.
5. Crea `Vendedor` (salario + comisión) y `Administrativo` (salario fijo), implementando ambas versiones.
6. En el `main`, guárdalos en un arreglo `Empleado[]` y muestra el salario de cada uno.

Reto extra: convierte `Administrativo` en `final class` y explica en un comentario qué consecuencia tiene para el futuro del sistema.

Autoevaluación final

1. ¿Qué impide **final** en una clase?

- ☐ a) Instanciarla
- ☐ b) Heredar de ella
- ☐ c) Usar sus métodos

2. Una clase abstracta...

- ☐ a) No se puede instanciar
- ☐ b) No puede tener constructor
- ☐ c) No puede tener métodos con cuerpo

3. ¿**abstract final** es válido?

- ☐ a) Sí, siempre
- ☐ b) Solo en clases
- ☐ c) Nunca, son contradictorios

4. **final List<String> l** — ¿puedes usar **l.add("x")**?

- ☐ a) Sí
- ☐ b) No
- ☐ c) Solo si es static

5. Para sobrecargar debes cambiar...

- ☐ a) Solo el retorno
- ☐ b) Los parámetros
- ☐ c) El nombre

☒ Solucionario

[▶ Ver respuestas](#)

Ya sabes qué sellar y qué dejar abierto

Dominas **final** en propiedades, métodos y clases; entiendes por qué una clase abstracta no se instancia y cómo un método abstracto obliga a las hijas a cumplir un contrato. Esa es la base del diseño orientado a objetos serio.

Próxima guía

Interfaces y polimorfismo